

Convolutional networks

Filters as sliding dot products, output size, padding and stride, pooling, channels, LeNet, and 1-D convolution.

THE QUESTION

Images have millions of pixels, and a cat's ear can sit anywhere in the frame. How do we build a network with few weights that finds a pattern wherever it appears?

THE INTUITION

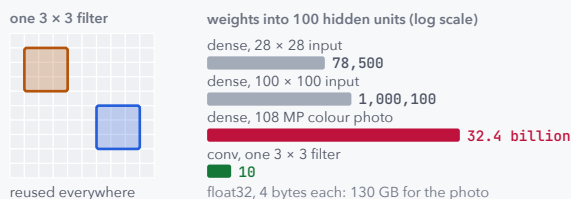
Slide one small filter over the image and take a dot product at each position. A 3×3 filter reuses its 9 weights everywhere, so it finds its pattern (an edge, a corner) wherever it is. Pooling and more layers then cover larger regions.

KEY TAKEAWAY

A conv layer has $c_{\text{out}}(f^2 c_{\text{in}} + 1)$ parameters and output side $\lfloor (n + 2p - f)/s \rfloor + 1$. Conv, ReLU and pooling, repeated, then dense layers: that is LeNet.

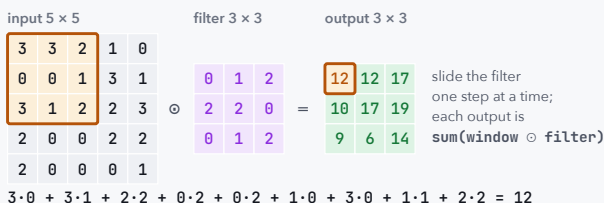
image $n \times n \times c$ → conv: $f \times f$ filters → ReLU → pool 2×2 → repeat → flatten → dense, softmax

1 Dense layers on pixels waste weights



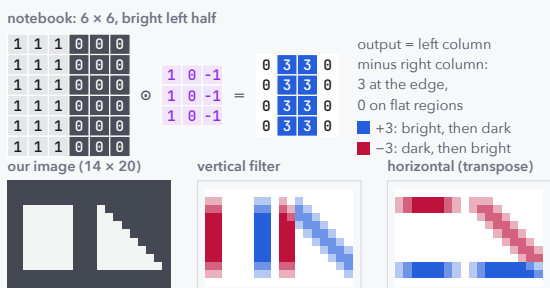
A 108-megapixel colour photo is 3.24×10^8 inputs; 100 hidden units need 32.4 billion weights. An MLP also treats an ear at the top left and the same ear lower down as unrelated inputs, and ignores that neighbouring pixels are alike. A conv layer uses small local detectors and reuses them.

2 Convolution is a sliding dot product



Multiply the $f \times f$ filter with a patch element-wise and add; slide one pixel and repeat: $(n - f + 1)^2$ outputs. Libraries compute this cross-correlation; true convolution flips the filter first, which for $[1, 0, -1]$ flips the sign. Training learns the filter.

3 An edge filter fires where brightness changes



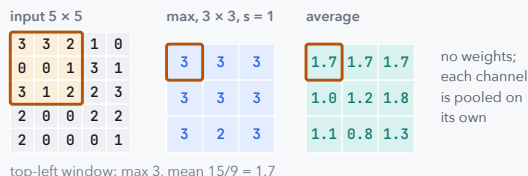
$[1, 0, -1]$ in every row subtracts the right column from the left: 0 on flat regions, ± 3 at a vertical edge, the sign telling which side is bright. The transpose finds horizontal edges; our triangle's staircase edge lights up both.

4 Output side: $\lfloor (n + 2p - f)/s \rfloor + 1$



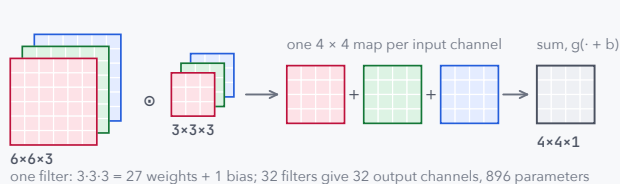
Padding p adds zeros around the border; stride s moves the filter s pixels at a time. "Same" padding, $p = (f - 1)/2$ with $s = 1$, keeps the side, so f is usually odd. Without it each layer trims $f - 1$: 32×32 through repeated 5×5 filters goes 28, 24, ..., 4 and never reaches 1×1 . Borders fade too: a corner pixel enters one window, the centre of a 5×5 input nine.

5 Pooling summarises a window without weights



Max pooling keeps the largest value in each window, average pooling the mean. The notes slide 3×3 with stride 1; LeNet uses $f = s = 2$, halving each side. Pooling cuts later computation and makes the output change less when a pattern shifts by a pixel.

6 A filter spans every input channel



On an $n \times n \times c_{\text{in}}$ input a filter is $f \times f \times c_{\text{in}}$: each channel meets its own slice, the maps are summed with one bias and passed through g (ReLU). c_{out} filters give c_{out} channels. The notebook's $3 \times 3 \times 3$ edge filter turns a $32 \times 32 \times 3$ image into $30 \times 30 \times 1$.

Sharing, receptive fields, LeNet and 1-D signals

Parameter counts against dense layers, what a unit sees, a full LeNet stack, training, and convolution over time.

7 Sharing cuts parameters by orders of magnitude

parameters for the same input and output shape (log scale)

LeNet C1: $32 \times 32 \times 1$ to $28 \times 28 \times 6$

conv 156

dense 4,821,600

100x100 to 98×98 , one 3×3 filter

conv 10

dense 96,049,604

$32 \times 32 \times 3$ to $32 \times 32 \times 32$, same padding

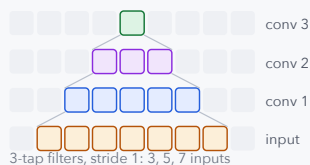
conv 896

dense 100,696,064

the conv layer reuses its filters at every position and sees only an $f \times f$ patch

A dense layer with the same input and output shapes needs a weight per input-output pair: 4.8 million for LeNet's first layer against 156. A conv layer is a dense layer with most weights forced to 0 (locality) and the rest tied across positions (sharing). The counting rule of the multi-layer perceptrons sheet still holds for the dense layers at the end.

8 Deeper units see larger regions



LeNet (notes), 32×32 input

layer	sees	step j
C1	5×5	1
S2	6×6	2
C3	14×14	2
S4	16×16	4

$r += (f - 1) \cdot j, j += s$

A unit's receptive field is the input patch that can change it. Stacked 3×3 convs grow it by 2 per layer; a stride or pool multiplies the growth of every later layer. In the notes' LeNet a C3 unit sees 14×14 of the 32×32 input. Early layers find edges; later ones combine them into parts.

9 A convolution layer in NumPy

```
def conv2d(x, k, s=1, p=0):
    x = np.pad(x, p) # zeros
    f = k.shape[0]
    n = (x.shape[0] - f) // s + 1
    out = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            win = x[i*s:i*s+f, j*s:j*s+f]
            out[i, j] = (win * k).sum()
    return out
```

A full layer loops over filters and sums over channels, as `nn.Conv2d` does.

13 Pause and think

$n = 64, f = 3, p = 1, s = 2$? Side 32.
Same padding for $f = 7$? $p = 3$.

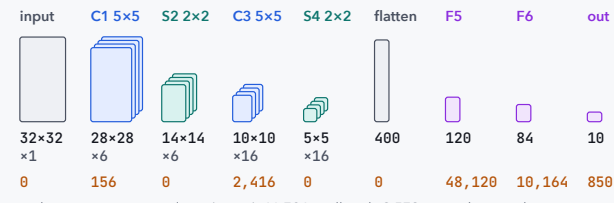
Ten 5×5 filters on RGB? 760.
Two stacked 3×3 convs see? 5×5 .

Weights in a 2×2 max pool? None.
Stride 2 on a 28 side, $f = 2$? 14.

sliding dot product $\cdot \lfloor (n + 2p - f) / s \rfloor + 1 \cdot c_{\text{out}}(f^2 c_{\text{in}} + 1)$ weights $\cdot$ pool to shrink $\cdot$ conv blocks, then dense

Before: the next-character prediction sheet; next: the unsupervised learning sheet. Source: lecture notes "Convolutional neural networks" and the convolution-operation, convolution-stride, CNN edge, CNN, LeNet and 1-D CNN notebooks (nipunbatra.github.io/ml-teaching). The 14×20 image and the dense comparisons are ours. Corrected: the notes' 108 MP example has 32.4 billion weights, not 324 billion, at 4 bytes (not 32) each; the 1-D notebook's "silence" filter $[-1, -1, -1]$ fires on the -1 stretches.

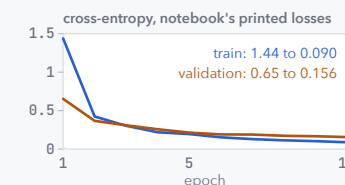
10 LeNet-5: two conv blocks, then dense layers



amber: parameters per layer (notes), 61,706 in all; only 2,572 are in the conv layers

Sides $32 - 5 + 1 = 28$, pool to 14, $14 - 5 + 1 = 10$, pool to 5; flatten 400. Weights $6 \cdot 25 + 6 = 156$, $16 \cdot 150 + 16 = 2416$, then 48120, 10164 and 850 (each C3 filter reads all 6 maps). The Keras notebook's version (3×3 filters, average pooling, 28×28 digits) has 60,074 and gets 98.8% on test.

11 Training: the same loop as any network

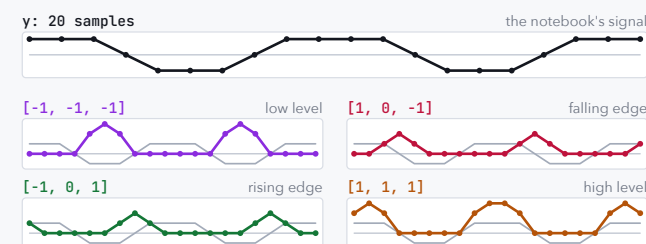


one digit through the net

conv1, ReLU	$6 \times 24 \times 24$
max pool 2	$6 \times 12 \times 12$
conv2, ReLU	$16 \times 8 \times 8$
max pool 2	$16 \times 4 \times 4$
flatten	256
dense	120, 84, 10

The PyTorch notebook trains a 44,426-parameter LeNet on 5,000 digits with cross-entropy and Adam: 96.1% test accuracy. For a new task, train from scratch or put a new dense head on trained conv layers (transfer learning).

12 1-D convolution slides along time



ReLU(filter $\odot$ window), padding 1, so 20 outputs; grey: the input y

`nn.Conv1d(1, 1, 3, padding=1)`, hand-set filters: $[1, 0, -1]$ gives $y_{t-1} - y_{t+1}$, positive on falling edges; $[-1, 0, 1]$ marks rising ones; $[1, 1, 1]$ (three times a moving average) high stretches; $[-1, -1, -1]$ low ones. Zero padding adds an edge at each end.